VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE

VJTI, MUMBAI

OPTIMIZE ANYTHING

The 14-Day Vibe-Coding Challenge

Optimization Techniques Lab Assignment 8

Submitted by

Bianca Sawant

Registration No. 241071067

Course: Optimization Techniques

Academic Year: 2026–2027

Evidence-based report of Tracks A–J

Contents

1 Introduction.....5
2 Experimental Environment and Shared Method.....6
3 Cross Track Method Selection.....7
4.A Track A Continuous Optimization.....8
4.B Track B Discrete Optimization.....10
4.C Track C Quantum Optimization.....12
4.D Track D Submodular, Parallel, and Distributed Optimization.....14
4.E Track E Inference Request Batching and Scheduling.....16
4.F Track F Toy Language Compiler Optimization.....19
4.G Track G xv6 Priority Scheduling.....21
4.H Track H SQLite Index Selection.....23
4.I Track I Quantum Algorithms and Quantum Machine Learning.....25
4.J Track J Software Engineering Test Selection.....27
5 Cross Track Analysis.....28
6 AI Assisted Development and Verification.....30
7 Conclusion.....31
References.....31
Appendix A Chapter 6 Coverage Checklist.....32
Appendix B Reproduction Commands and Evidence Map.....33

List of Figures

Figure 1. Evidence workflow used across the ten tracks.....5
Figure 2. Track A measured comparison retained in the repository results.....9
Figure 3. Track B measured comparison retained in the repository results.....11
Figure 4. Track C measured comparison retained in the repository results.....13
Figure 5. Track D measured comparison retained in the repository results.....15
Figure 6. Track E measured comparison retained in the repository results.....18
Figure 7. Track F measured comparison retained in the repository results.....20
Figure 8. Track G measured comparison retained in the repository results.....22
Figure 9. Track H measured comparison retained in the repository results.....24
Figure 10. Track I measured comparison retained in the repository results.....26
Figure 11. Hand-constructed Deutsch–Jozsa, Grover and QFT circuits from Track I.....26
Figure 12. Track J measured comparison retained in the repository results.....28

List of Tables

Table 1. Machine and reproducibility profile....6
Table 2. Repository-wide method selection....7
Table 3. Continuous method comparison on the regularized logistic objective....9
Table 4. Selection solution quality and exactness....10
Table 5. Quantum optimization results and simulation cost....13
Table 6. Measured process scheduling and parallel-form outcomes....14
Table 7. Track E strategy selection....17
Table 8. Selected 128-request scheduling outcomes at token capacity 512....17
Table 9. Compiler size and timing across five programs....19
Table 10. Priority scheduler result by workload....21
Table 11. SQLite workload performance and index storage....23
Table 12. Matched Iris quantum and classical model results....26
Table 13. Test selection under a 0.044501 s profiled budget....27
Table 14. Cross-track findings and limits....29
Table 15. AI tool disclosure and provenance status....30
Table 16. Coverage of methods expressly named by the assignment PDF....32

1 Introduction

Assignment 8 asks us to treat optimization as a transferable problem-solving method. The work therefore focuses on identifying a process, defining decision variables and measurable objectives, applying suitable methods, checking correctness independently, and comparing each result with a fair baseline. Tracks A–D exercise the optimization topics; Track E is the selected process case study; Tracks F–J apply optimization to a compiler, operating system, database, quantum computing and software engineering.

The project deliberately uses small benchmarks that can be rerun locally on a 16 GB MacBook Air. Small instances make exact enumeration and invariant checks feasible; they do not establish production scalability. The evidence distinguishes mathematical or query-complexity statements from CPU measurements, simulator measurements and QEMU measurements. Negative speedups, exhausted stopping budgets, equal model accuracy and unresolved course-specific methods are retained as findings rather than edited away.

1.1 Objectives and workflow

For each experiment the workflow records the baseline, formal model, method choice, input identity, constraints, repetitions, correctness checks and measured outcome. The shared workflow used across tracks is shown below. Domain-specific implementations remain separate even where they reuse helpers, so a result from one track is not counted as evidence for another.

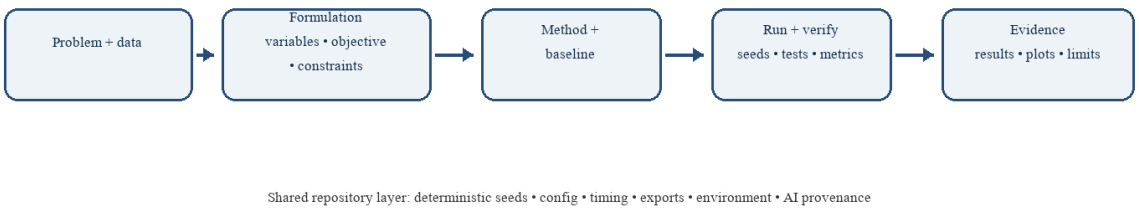


Figure 1. Evidence workflow used across the ten tracks.

1.2 Scope and completion statement

All ten tracks have executable implementations, tests, retained results and report sections. The assignment is not fully closed against the classroom syllabus: the actual Chapter 6 lecture

notes or approved topic list are absent, leaving conditional coverage in A–D and I unresolved. The AI register shows one tool, OpenAI Codex; the required comparison between two AI tools has not occurred. The report describes both as open requirements. No results are invented to fill those gaps.

2 Experimental Environment and Shared Method

The measured development machine is a MacBook Air, model identifier Mac17,3, with an Apple M5, arm64 architecture, 10 CPU cores (4 performance and 6 efficiency), 16 GB RAM, macOS 26.5.2 and approximately 288 GiB free disk after setup. The isolated project environment uses CPython 3.12.13. CPU is the reproducible PyTorch baseline; MPS was compiled into the build but unavailable at verification time. No dependency was installed into the system Python.

The direct dependency set is NumPy 2.5.4, SciPy 1.18.1, matplotlib 3.11.2, scikit-learn 1.9.1, PyTorch 2.15.0, PennyLane 0.45.1, NetworkX 3.7, pytest 9.2.0 and pytest-cov 7.1.0. `uv.lock` pins transitive dependencies. SciPy/HiGHS supplies the integer solver; PennyLane supplies local circuit simulation. SQLite, the macOS spawn multiprocessing context, and RISC-V tools are checked in preflight. QEMU 11.1.1 and RISC-V GCC 16.2.0 support Track G.

Shared repository infrastructure centralizes random seeds, TOML configuration, timing and peak-memory measurements, repeated runs, JSON/CSV export, plot generation, invariant checks, environment metadata and the track registry. Prompts, model disclosure, verification notes and commits are archived under prompts/. Raw measurements remain alongside summaries and figures. Three repeats are used where feasible; data is held constant across method comparisons.

Table 1. Machine and reproducibility profile

Item	Verified setup
Computer	MacBook Air, Apple M5, arm64, 10 cores, 16 GB RAM
Operating system	macOS 26.5.2; Darwin 25.5.0
Python	CPython 3.12.13 in isolated .venv
Numerical/ML/quantum	NumPy, SciPy/HiGHS, PyTorch CPU, PennyLane
Native OS tools	QEMU 11.1.1; riscv64-elf GCC 16.2.0
Environment gate	7 of 7 preflight checks passed

The final preflight checks real operations: numerical optimization, tensor autograd, binary ILP, quantum-circuit execution, SQLite plans, spawned workers and RISC-V compile/disassembly. `SETUP.md` records installation decisions, real command outputs, and an observed Homebrew post-install cleanup side effect; this report does not imply that QEMU tools were present before setup.

3 Cross Track Method Selection

Method choice follows variable type, constraints, problem size, available hardware, convergence or optimality needs, and communication cost. Exact enumeration is used as a correctness oracle only where the state space is deliberately small. Heuristics are evaluated against those references without assuming that they are optimal.

Table 2. Repository-wide method selection

Track	Decision variables / constraints	Chosen comparison	Hardware / communication
A	Continuous weights; regularized loss	First-order, second-order, curvature approximation	CPU; no communication
B	Binary item selection; capacity	Greedy vs ILP, B&B, DP, enumeration	CPU; exact oracle
C	Circuit angles; Max-Cut energy	QAOA, VQE, SPSA, QNG, shots/noise	CPU simulator; circuit evaluations
D	Sets, tasks, model parameters	Greedy/exact, process scheduling, FedAvg, MGDA, GradNorm	Spawned CPU processes; IPC measured
E	Request-to-batch assignments; latency/token/SLO limits	FCFS vs greedy vs local DP; exact tiny oracle	Single-server simulation
F	Ordered optimization passes; effort budget	Four passes; B&B vs exhaustive order	CPU toy compiler
G	Runnable processes; bounded priorities	Table-scan baseline vs aging priority	One RISC-V CPU in QEMU
H	Binary optional-index choices; page budget	No-index vs greedy, MILP, DP	SQLite local CPU
I	Quantum algorithm and model parameters	Textbook circuits vs classical baselines	CPU simulator; no quantum hardware
J	Binary test selection; profiled time budget	Original order vs greedy; small exact solver	Public project CPU tests

4.A Track A Continuous Optimization

4.A.1 Problem, objective, and formulation

A minimizes regularized binary logistic loss on 160 synthetic two-feature observations. Batch gradient descent is the reference; eight first-order variants share the data, initialization, 60 passes, and three seeds. Newton and BFGS use the same logistic objective, while Gauss–Newton is tested on a nonlinear least-squares exponential fit.

For labels $y_i \in \{0,1\}$, features x_i and parameter w, $L(w)=n^{-1}\Sigma[\log(1+e^{x_i^T w})-y_i x_i^T w]+(\lambda/2)\|w\|^2$, with $\lambda=0.1$. Backpropagation computes ∇L; an optimizer separately changes w. The synthetic convex objective, strongly convex quadratic, and tilted double-well landscape isolate curvature and non-convex behavior.

$$L(w)=1/n\ \Sigma_i[\log(1+e^{x_i^T w})-y_i x_i^T w]+(\lambda/2)\|w\|^2 \quad (1)$$

4.A.2 Algorithms and experimental method

First-order methods were batch GD, SGD, mini-batch SGD, Momentum, Nesterov, AdaGrad, RMSProp and Adam. Curvature methods included Newton, Gauss–Newton, BFGS, exact Hessian-vector products, diagonal Hessian, empirical Fisher and rank-1 approximation. A small PyTorch tanh network demonstrates backward gradient computation before the optimizer step. Central finite differences provide independent derivative checks.

4.A.3 Results and interpretation

Three-run first-order means ranged from 0.391126 (mini-batch) to 0.391433 (batch GD). All first-order runs exhausted the 60-pass budget before the strict 10^{-6} tolerance. Newton and BFGS reached about 0.391126; Gauss–Newton recovered planted parameters [1.5, −0.8]. The logistic gradient check's maximum absolute error was 7.38×10^{-11}; the XOR model reached 100% training accuracy. On the tilted double well, the global minimum was near (−1.024, 0), the local minimum near (0.974, 0), and the saddle near (0.0501, 0). Learning rate 0.22 diverged where the fixed-step bound was 0.2. Increasing λ reduced parameter norm $2.494 \rightarrow 0.368$ while accuracy changed $0.938 \rightarrow 0.931$; three noisy runs showed a nonzero oscillating error floor.

Table 3. Continuous method comparison on the regularized logistic objective

Method	Mean final objective	Mean runtime (s)	Status
Batch GD	0.391433	0.00821	budget exhausted
SGD	0.391133	0.39850	budget exhausted
Mini-batch	0.391126	0.03090	budget exhausted
Momentum / NAG	0.391156 / 0.391153	0.03582 / 0.03931	budget exhausted
AdaGrad / RMSProp / Adam	0.391127 / 0.391148 / 0.391192	0.04037 / 0.05099 / 0.07342	budget exhausted
Newton / BFGS	0.391126 / 0.391126	0.00080 / 0.00533	converged

4.A.4 Correctness, limitations, and reproduction

The experiments separate optimizer choice from gradient computation and distinguish objective value from runtime. SGD's 0.398 s Python runtime exceeded batch GD's 0.0082 s because it made many more updates; this is an implementation-cost result, not an algorithmic theorem. Diagonal/Fisher/low-rank curvature approximations had probe-vector errors 0.194/0.248/0.649, whereas exact HVP relative error was 4.90×10^{-11}. The actual course list is still required to determine whether more curvature methods must be added.

Reproduction: ./scripts/run_track.py A | .venv/bin/python -m pytest tracks/A_continuous/tests

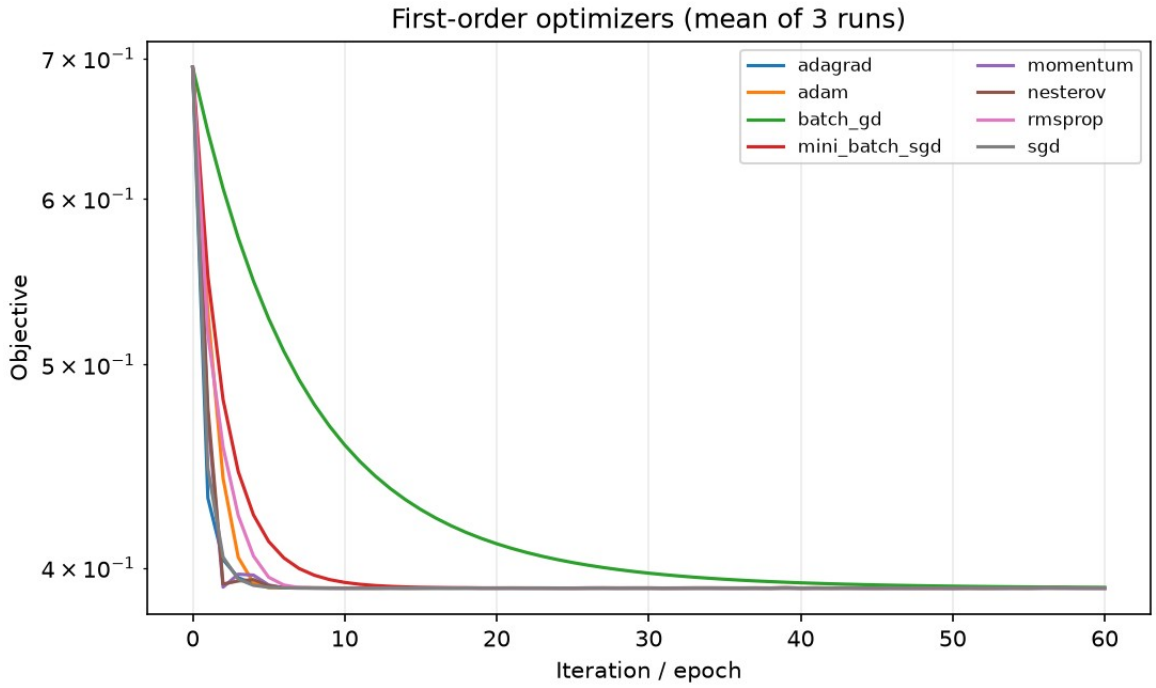


Figure 2. Track A measured comparison retained in the repository results.

4.B Track B Discrete Optimization

4.B.1 Problem, objective, and formulation

B maximizes value in a 0/1 selection problem subject to one capacity constraint. The study uses three fixed knapsack instances with 3, 8 and 14 items, and compares density greedy, binary ILP, branch-and-bound and dynamic programming.

Let $x_i \in \{0,1\}$ indicate selection, v_i be value, w_i be weight and C be capacity. Maximize $\Sigma v_i x_i$ subject to $\Sigma w_i x_i \leq C$. A separate weighted Max-Cut graph defines binary x_i for side membership. Its cut is $\Sigma w_{ij}(x_i+x_j-2x_i x_j)$, and the minimized QUBO is the negative cut. Under $s_i=2x_i-1$, the Ising offset is $-\Sigma w/2$ and pair coupling is $w_{ij}/2$. A separate exactly-one selection adds $A(\Sigma x_i-1)^2$ as a penalty.

$$\text{maximize } \Sigma_i v_i x_i, \text{ subject to } \Sigma_i w_i x_i \leq C,\ x_i \in \{0,1\};\ Q(x)=-\Sigma_(i,j) w_{ij}(x_i+x_j-2x_i x_j) \qquad (2)$$

4.B.2 Algorithms and experimental method

Greedy supplies a fast feasible lower bound; ILP, branch-and-bound and DP are exact on these instances. Exhaustive enumeration independently checks all selections and QUBO/Ising energies. Max-Cut uses the exact same hashed five-vertex fixture for Track C.

4.B.3 Results and interpretation

Greedy objectives were 160/220, 39/44 and 82/83 (gaps 27.27%, 11.36% and 1.20%). All three exact methods matched exhaustive optima. Greedy runtimes were about 21–34 µs; branch-and-bound about 40–100 µs; DP 0.30–1.46 ms; HiGHS ILP about 1.8–6.9 ms. The five-vertex graph has seven weighted edges and exact maximum cut 8.0, with four equivalent optimum bitstrings. A=5 was the smallest tested exactly-one penalty that made the ground state feasible.

Table 4. Selection solution quality and exactness

Instance	Greedy	Exact optimum	Greedy gap	Exact methods
3-item counterexample	160	220	27.27%	ILP = B&B = DP
8-item selection	39	44	11.36%	ILP = B&B = DP
14-item allocation	82	83	1.20%	ILP = B&B = DP

4.B.4 Correctness, limitations, and reproduction

For toy instances, solver startup dominates and greedy is fastest but can lose value. DP is pseudo-polynomial in capacity; branch-and-bound has exponential worst-case complexity; ILP supports richer constraints. The distinction between unconstrained Max-Cut (no penalty needed) and the separate constrained example avoids assigning an irrelevant penalty to the graph problem. Any additional class-specific discrete application remains syllabus-dependent.

Reproduction: ./scripts/run_track.py B | .venv/bin/python -m pytest tracks/B_discrete/tests

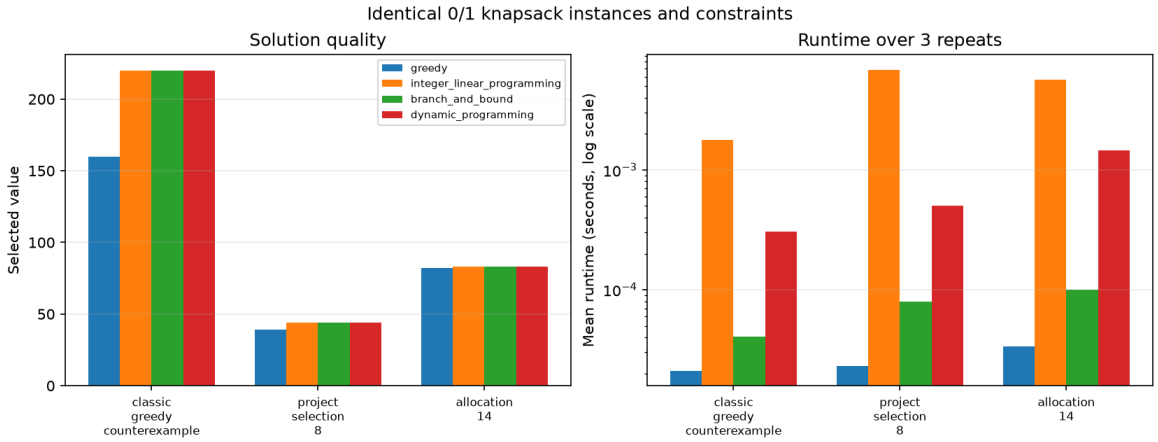


Figure 3. Track B measured comparison retained in the repository results.

4.C Track C Quantum Optimization

4.C.1 Problem, objective, and formulation

C applies hybrid quantum-classical optimization to the exact Max-Cut instance from Track B, then tests a small VQE Hamiltonian and a two-qubit parameterized quantum model. All outputs are from local CPU simulation.

For graph edges E, the cost operator is $H_C=-\Sigma_{(i,j)} w_{ij}(I-Z_iZ_j)/2$, whose expectation is negative cut value. QAOA alternates cost and mixer unitaries for depth p and optimizes angles to minimize $\langle H_C\rangle$. VQE minimizes $\langle\psi(\theta)|H|\psi(\theta)\rangle$. The parameter-shift derivative is $[f(\theta+\pi/2)-f(\theta-\pi/2)]/2$. SPSA estimates a stochastic gradient using two objective evaluations per iteration; quantum natural gradient preconditions by a circuit metric tensor.

$$H_C=-\Sigma_{(i,j)}w_{ij}(I-Z_iZ_j)/2;\ \text{minimize}_\theta\ \langle\psi(\theta)|H_C|\psi(\theta)\rangle \qquad (3)$$

4.C.2 Algorithms and experimental method

PennyLane statevector and mixed-state devices run QAOA, VQE, QNN, SPSA, parameter shift, QNG, finite-shot and depolarizing-noise studies. Exhaustive classical enumeration verifies all 32 Max-Cut basis energies. Circuit diagrams and quantum/classical reference values are retained in the track result folder.

4.C.3 Results and interpretation

Mean expected cut rose from about 6.09 at p=1 to 6.87 at p=3. The best run had expected cut 7.2723, ratio 0.909, and 49.4% probability on exact optima; the most likely bitstring is optimal. Twelve-shot repetitions reduced mean absolute estimator error from 0.0988 at 100 shots to 0.00954 at 10,000. Reference noise lowered expected cut from 7.2723 to 6.8040; double noise lowered it to 6.4295. QNG reached the exact two-qubit VQE energy within 2.2×10^{-14} on three seeds. QNN and classical baseline both fit four training points perfectly, but the classical model took about 0.0023 s versus about 0.85 s for QNN simulation. In the embedding proxy, chain strengths 0.25, 1.0 and 3.0 left 100%, 25% and 0% of physical ground states chain-broken.

Table 5. Quantum optimization results and simulation cost

Experiment	Best / reference	Measured outcome	Interpretation
QAOA, p=3	Exact cut = 8.0	Expected 7.2723; 49.4% optimum probability	No quantum advantage
Finite shots	100 to 10,000 shots	Error 0.0988 to 0.00954	More shots reduce estimation error
VQE + QNG	Exact energy −1.0369316877	Error ≤2.2×10^{-14} on 3 seeds	Two-qubit simulation
QNN vs classical	Both 100% train accuracy	≈0.85 s vs ≈0.0023 s	Classical baseline faster

4.C.4 Correctness, limitations, and reproduction

The exact classical solver wins decisively on this five-node graph. A finite-size global-cost gradient variance decline from 0.285 at two qubits to 9.03×10^{-4} at ten is a barren-plateau warning, not proof of asymptotic scaling. Circuit calls, shots and noise are part of the cost. No result is from a quantum device, and no quantum advantage is claimed.

Reproduction: ./scripts/run_track.py C | .venv/bin/python -m pytest tracks/C_quantum_optimization/tests

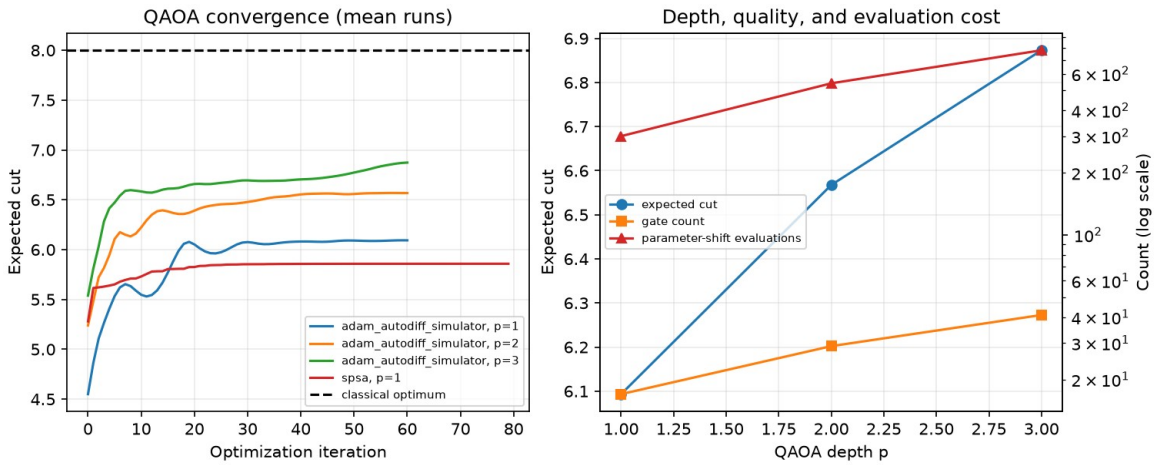


Figure 4. Track C measured comparison retained in the repository results.

4.D Track D Submodular, Parallel, and Distributed Optimization

4.D.1 Problem, objective, and formulation

D tests submodular maximum coverage, process parallel scheduling, three parallelization forms, simulated federated averaging, and two-task loss balancing. The same deterministic workloads are checked against serial or exhaustive references.

Coverage maximizes $f(S)=|\bigcup_{j\in S}A_j|$ subject to $|S|\le k$. For differentiable shared-model tasks, FedAvg averages local models with client-size weights. The minimum-norm two-task MGDA rule chooses $\alpha\in[0,1]$ minimizing $\|\alpha g_1+(1-\alpha)g_2\|^2$; GradNorm adapts positive task weights to balance weighted gradient norms and relative training rates. Parallel efficiency is $E_p=T_1/(pT_p)$, while speedup is T_1/T_p.

$$f(S)=|\bigcup_{j\in S}A_j|,\ |S|\le k;\ \text{speedup}=T_1/T_p,\ \text{efficiency}=T_1/(pT_p) \qquad (4)$$

4.D.2 Algorithms and experimental method

The set function was checked exhaustively for monotonicity and diminishing returns. Twelve CPU-bound tasks were run serially, with contiguous static chunks, and through a process-shared queue under macOS spawn. Work stealing was also demonstrated in a deterministic event simulation. Separate data-, model- and pipeline-parallel calculations used independently spawned processes and exact-output checks.

4.D.3 Results and interpretation

Greedy covered 7/10 elements while exhaustive optimum covered 8/10 (ratio 0.875). Static scheduling produced 0.697× speedup at two workers and 0.846× at four. The real shared queue achieved 1.065× median speedup at four workers, with 26.6% efficiency; repeats included slowdowns. Data/model/pipeline parallel speedups were 0.00139, 0.000706 and 0.000810 because roughly 0.1 s process startup dwarfed 0.0001 s serial kernels. FedAvg with one local step matched central GD exactly; at three local steps its 10-round loss was 0.33380 versus central 0.33345. Equal task weighting had mean loss 0.3977, GradNorm 0.4032, and this MGDA run 1.2007 with a strong task imbalance.

Table 6. Measured process scheduling and parallel-form outcomes

Configuration	Median time (s)	Speedup	Efficiency
Serial	0.1953	1.000	—

Configuration	Median time (s)	Speedup	Efficiency
Static, 2 workers	0.2800	0.697	0.349
Static, 4 workers	0.2309	0.846	0.211
Shared queue, 2 workers	0.2173	0.899	0.449
Shared queue, 4 workers	0.1835	1.065	0.266
Data / model / pipeline	startup dominated	0.00139 / 0.000706 / 0.000810	negative speedup

4.D.4 Correctness, limitations, and reproduction

The coverage guarantee 1−1/e applies only to normalized nonnegative monotone submodular objectives under cardinality constraints. Amdahl inversion returned an impossible raw serial fraction 1.225 under the two-worker slowdown, revealing that startup overhead violates the ideal model. Work stealing balances load but cannot make tasks large enough to amortize process overhead. MGDA did not dominate equal weighting on this two-task sample. No network federation or privacy guarantee is simulated.

Reproduction: ./scripts/run_track.py D | .venv/bin/python -m pytest tracks/D_parallel_distributed/tests

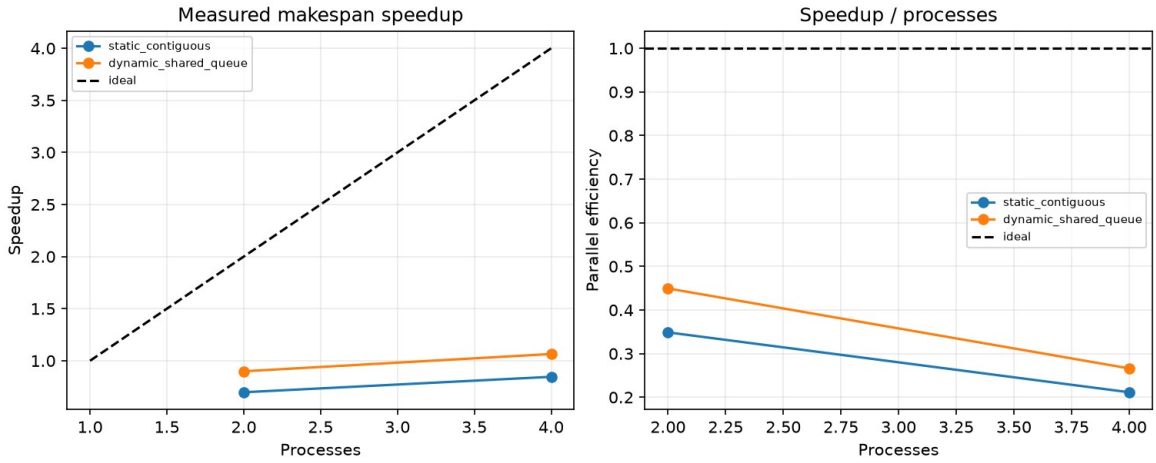


Figure 5. Track D measured comparison retained in the repository results.

4.E Track E Inference Request Batching and Scheduling

4.E.1 Problem, objective, and formulation

E studies inference-request batching using a reproducible discrete-event simulation of a single server. Requests have seeded arrivals, token lengths, costs and SLO deadlines. No real model, accelerator, energy use or production inference latency was measured.

Binary assignment variables map each request to exactly one ordered batch; all batches obey request-count and token/padded-token capacities, arrival precedence, single-server non-overlap, and bounded formation wait. Deadline violations use explicit nonnegative tardiness slack. For batch b, modeled service is $T_b=0.35+\max(p_i)[1+0.22(n_b-1)]$. The objective combines mean latency, twice mean tardiness and 0.15 busy-time/request. This is a stated simulation model, not measured hardware service.

$$T_b=0.35+\max_{(i\in b)}(p_i)[1+0.22(|b|-1)] \quad (5)$$

4.E.2 Algorithms and experimental method

Immediate FCFS is compared with bounded-window length-aware greedy and dynamic programming that exactly solves a local count/token/urgency score over a 12-request lookahead. Three workload sizes (24, 64, 128), two arrival rates, two token capacities and three seeds share workload digests across methods. A six-request exhaustive ordered-partition reference checks the local/global distinction.

4.E.3 Results and interpretation

At 128 requests and rate 1.2, capacity 512, FCFS mean/p95 latency were 12.911/24.390, throughput 1.025 requests per simulated unit, makespan 124.986, utilization 0.994 and deadline-miss rate 73.4%. Greedy reduced these to mean/p95 7.116/15.992, makespan 113.435 and 47.9% misses while throughput rose to 1.131; modeled utilization was 0.956. At sparse rate 0.6, FCFS mean latency/wait were 3.186/1.016, compared with greedy 3.914/1.475 and DP 4.023/1.518, with similar throughput near 0.585. Greedy token capacity $384\rightarrow512$ improved loaded throughput $1.084\rightarrow1.131$, while miss rate shifted only $48.2\%\rightarrow47.9\%$.

Table 7. Track E strategy selection

Strategy	Variables / scale	Exact or heuristic	Hardware / parallelism
FCFS	Ordered requests; all sizes	Online baseline	Single simulated server; one event loop
Length-aware greedy	Binary request-to-batch choice; 24–128 requests	Bounded-window heuristic	CPU discrete-event simulation
Dynamic programming	Count/token states; 12-request lookahead	Exact local score, not global schedule	CPU; sequential DP
Exhaustive reference	Six simultaneous requests	Exact ordered-partition oracle	CPU; tiny instance only

Table 8. Selected 128-request scheduling outcomes at token capacity 512

Arrival rate	Strategy	Mean latency	p95 latency	Throughput	Miss rate
0.6	FCFS	3.186	—	≈0.585	—
0.6	Greedy	3.914	—	≈0.585	—
0.6	DP	4.023	—	≈0.585	—
1.2	FCFS	12.911	24.390	1.025	73.4%
1.2	Greedy	7.116	15.992	1.131	47.9%
1.2	DP	8.936	17.166	1.102	68.5%

4.E.4 Correctness, limitations, and reproduction

Waiting to form batches can increase latency when arrivals are sparse. Under load, batching improves the simulator's throughput and SLO rate, but service-time and padding assumptions determine those results. The local DP score is not a globally optimal schedule objective; the tiny exact reference found 2.5778, while DP's complete schedule scored 2.5932. Multi-server routing, KV-cache behavior, real token-generation dynamics and prediction error are outside scope.

Reproduction: ./scripts/run_track.py E | .venv/bin/python -m pytest tracks/E_case_study/tests

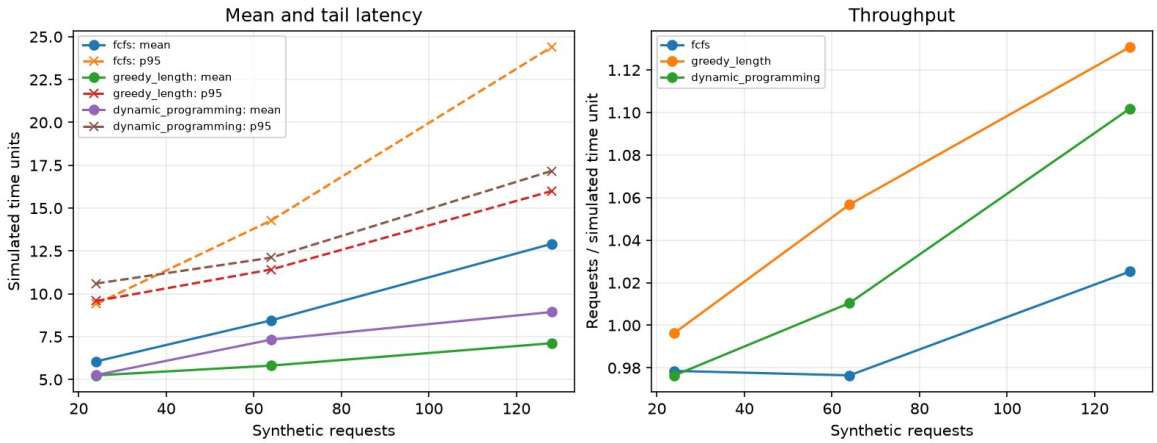


Figure 6. Track E measured comparison retained in the repository results.

4.F Track F Toy Language Compiler Optimization

4.F.1 Problem, objective, and formulation

F implements a small integer toy language with declarations, reassignment, arithmetic and observable print statements. It is a lexer-parser-IR-interpreter pipeline, not a native-code compiler.

The decision experiment chooses an ordered subset of compiler passes to minimize total static IR instruction count subject to nominal optimization effort ≤8 and at most one use of each pass. The full compiler separately applies four semantics-preserving transformations: constant folding/propagation, common subexpression elimination, strength reduction for positive powers of two, and dead-code elimination.

$$\text{minimize } I(\pi), \text{ subject to effort}(\pi) \le 8 \text{ and each pass used at most once} \tag{6}$$

4.F.2 Algorithms and experimental method

The implementation uses source-offset lexing, recursive-descent parsing, SSA-like three-address IR and a deterministic interpreter with truncation-toward-zero division. Five programs cover constants, repeated expressions, strength-reduction candidates, reassignment, multiple inputs and outputs. A branch-and-bound pass-order solver is checked against exhaustive enumeration of all 35 feasible ordered choices.

4.F.3 Results and interpretation

The four passes reduced suite IR instructions from 42 to 23 (45.2%) and serialized IR from 587 to 308 bytes (47.5%). Mean interpreter time sum decreased $3.947 \rightarrow 2.383$ µs (39.6%), while compilation time rose $97.36 \rightarrow 149.64$ µs (53.7%). The budgeted branch-and-bound choice selected folding/propagation followed by DCE at effort 6, reducing instructions $42 \rightarrow 25$; exhaustive enumeration agreed. All five optimized outputs match baseline and expected values.

Table 9. Compiler size and timing across five programs

Measure	Baseline	Optimized	Change
Static IR instructions	42	23	−45.2%
Serialized IR bytes	587	308	−47.5%
Sum interpreter time	3.947 µs	2.383 µs	−39.6%

Measure	Baseline	Optimized	Change
Sum compilation time	97.36 µs	149.64 µs	+53.7%

4.F.4 Correctness, limitations, and reproduction

The benchmark separates code-size and static-instruction effects from noisy microsecond interpreter timing. Python does not execute the IR as native machine code, and fewer IR instructions do not establish hardware speed. Four passes add compilation work. The bounded language omits control flow, types beyond integers, functions and register allocation.

Reproduction: ./scripts/run_track.py F | .venv/bin/python -m pytest tracks/F_compiler/tests

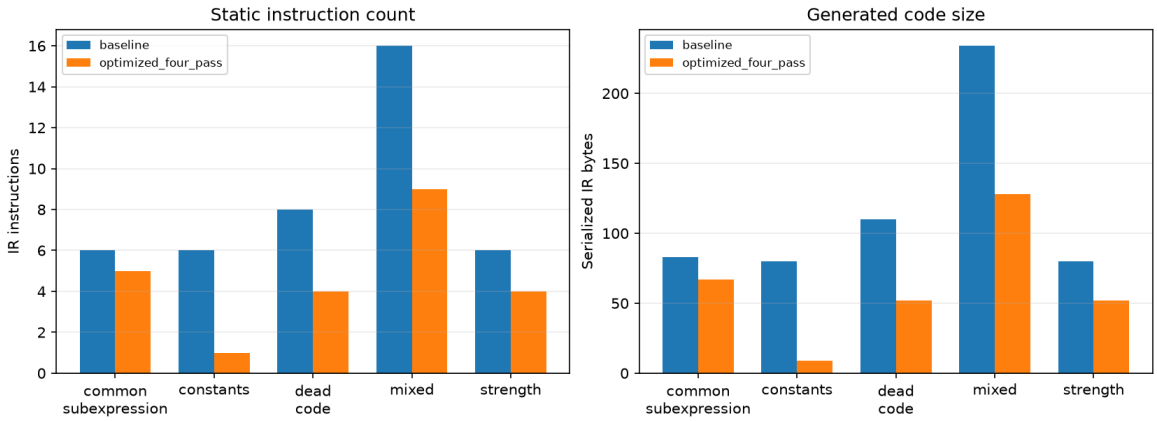


Figure 7. Track F measured comparison retained in the repository results.

4.G Track G xv6 Priority Scheduling

4.G.1 Problem, objective, and formulation

G changes one feature in pinned MIT xv6-riscv: a priority scheduler with aging. The measurement platform is a one-CPU RISC-V virtual machine under QEMU; it does not characterize the macOS scheduler or physical RISC-V hardware.

The objective minimizes high-priority job turnaround while retaining work conservation and low-priority completion. Priority is 0, 1 or 2, lower being higher; after eight runnable scheduler rounds, waiting processes improve one effective level. The original scheduler scans the process table in slot order and runs runnable processes as timer preemption returns them to the scheduler.

$$\text{effective_priority}_i = \max(0, \text{base_priority}_i - \lfloor \text{wait_rounds}_i / 8 \rfloor) \quad (7)$$

4.G.2 Algorithms and experimental method

The patch adds bounded priority setting and scheduler selection. Reads and writes of process state, priority and waiting rounds remain under each p->lock. Only RUNNABLE processes are candidates, selection changes exactly one to RUNNING before context switch, and only the chosen process lock crosses swtch. Three identical workloads—CPU, timer-sleep I/O surrogate and mixed—spawn six jobs behind a start barrier. Baseline and priority images carry the same benchmark and syscall support.

4.G.3 Results and interpretation

For CPU-bound jobs, high-priority mean turnaround fell from 6.0 to 3.0 ticks; low-priority mean shifted 6.89 to 6.78 and every job completed. Makespan changed $7.33 \rightarrow 7.0$ ticks and throughput $0.821 \rightarrow 0.857$ jobs/tick. Jain's inverse-turnaround fairness fell $0.993 \rightarrow 0.870$, exposing inequality. I/O-surrogate and mixed results were unchanged at 30 and 10 ticks respectively. Pristine, support-baseline and optimized kernels each booted and passed full usertests; 18 workload boots completed all 108 jobs.

Table 10. Priority scheduler result by workload

Workload / metric	Baseline	Priority	Interpretation
CPU high-priority turnaround	6.0 ticks	3.0 ticks	−50%

Workload / metric	Baseline	Priority	Interpretation
CPU low-priority turnaround	6.89 ticks	6.78 ticks	all complete
CPU Jain fairness	0.993	0.870	lower equality
I/O surrogate	30 ticks	30 ticks	unchanged
Mixed workload	10 ticks	10 ticks	unchanged

4.G.4 Correctness, limitations, and reproduction

The data supports faster turnaround for high-priority CPU-bound jobs, not a general throughput gain. Tick resolution is coarse; the I/O test uses sleeps, not disk operations. Aging prevented starvation in these finite workloads but does not prove a universal bound. The scheduler-only patch is 75 lines and applies cleanly to the pinned upstream commit; the optional second kernel feature was not added.

Reproduction: ./scripts/run_track.py G | .venv/bin/python -m pytest tracks/G_xv6/tests

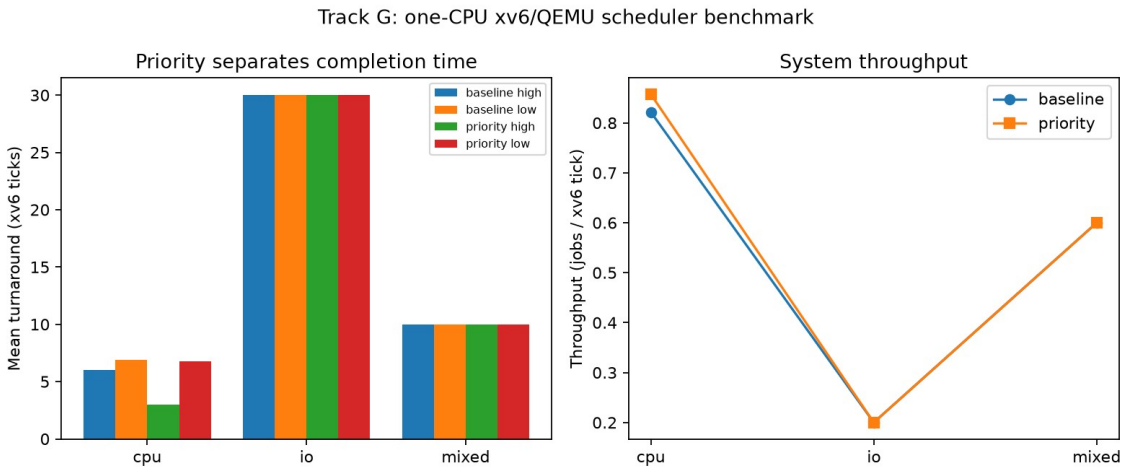


Figure 8. Track G measured comparison retained in the repository results.

4.H Track H SQLite Index Selection

4.H.1 Problem, objective, and formulation

H selects optional SQLite indexes under a measured 180-page storage budget. The synthetic relational dataset has 69,541 rows across customers, products, orders and order items, and five weighted parameterized queries.

For optional candidate index i, binary x_i indicates selection, s_i is measured page growth, and b_i is positive weighted latency reduction credited only when EXPLAIN QUERY PLAN uses that index. Minimize $C_0 - \Sigma b_i x_i$ subject to $\Sigma s_i x_i \leq 180$ and $x_i \in \{0,1\}$. This is an additive selection surrogate; final database performance is measured afresh because indexes may interact.

$$\text{minimize } C_0 - \Sigma_i b_i x_i, \text{ subject to } \Sigma_i s_i x_i \leq 180,\ x_i \in \{0,1\} \quad (8)$$

4.H.2 Algorithms and experimental method

The baseline has no optional indexes; alternatives are benefit-per-page greedy, binary MILP using SciPy/HiGHS, and independent exact 0/1 DP. Every strategy uses the same database and workload. Each configuration is rebuilt fresh; query results are hashed, EXPLAIN QUERY PLAN is captured, and each query has three warmups plus 15 timing repetitions over three full runs.

4.H.3 Results and interpretation

Baseline weighted mean latency was 0.420 ms (2,379 queries/s). Greedy used 136 pages/557,056 bytes and reached 0.0950 ms (10,529 queries/s), 77.4% lower latency. MILP and DP chose the same two indexes at exactly 180 pages; measured latency was 0.0493 and 0.0495 ms, 20,291 and 20,188 queries/s. All five logical-result hashes matched baseline under all four configurations and repeats. SQLite query plans showed scans replaced by searches through the selected indexes.

Table 11. SQLite workload performance and index storage

Configuration	Pages	Weighted mean (ms)	Queries/s	Latency reduction
No optional indexes	0	0.420	2,379	baseline
Greedy	136	0.0950	10,529	77.4%
MILP	180	0.0493	20,291	88.3%

Configuration	Pages	Weighted mean (ms)	Queries/s	Latency reduction
Dynamic programming	180	0.0495	20,188	88.2%

4.H.4 Correctness, limitations, and reproduction

The experiment optimizes index availability around SQLite's own query optimizer; it does not replace that optimizer. Candidate-benefit estimates omit interactions, the workload is synthetic and read-heavy, and warm-cache laptop timing does not predict production. Index maintenance cost and write performance are not included.

Reproduction: ./scripts/run_track.py H | .venv/bin/python -m pytest tracks/H_dbms/tests

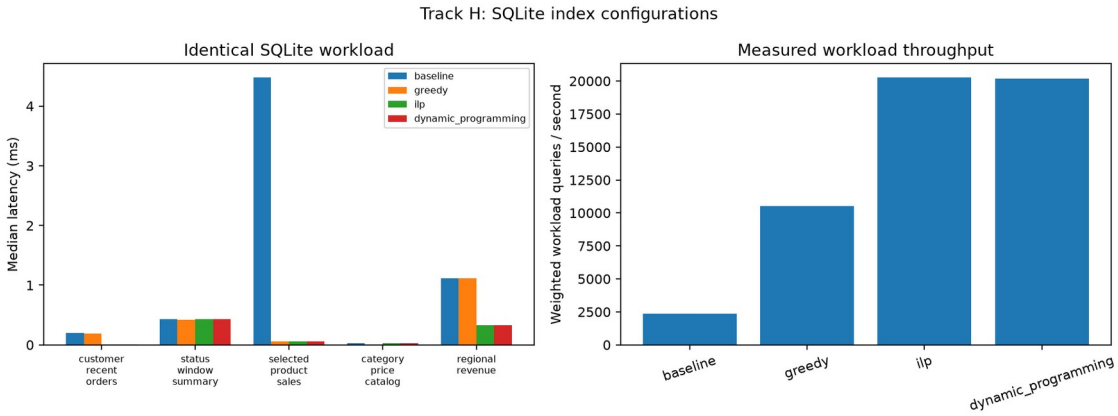


Figure 9. Track H measured comparison retained in the repository results.

4.I Track I Quantum Algorithms and Quantum Machine Learning

4.I.1 Problem, objective, and formulation

I executes the named small quantum-algorithm examples and compares quantum-model experiments with classical baselines. All circuits ran on a local CPU simulator; no quantum or annealing hardware was used.

The implemented algorithm set includes Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE, an N=15 Shor order-finding instance, and a classical simulated annealing proxy. Four circuits are built from primitive gates. QML uses an identical 30/10 Iris split for VQC/QNN, fidelity quantum kernel, logistic regression and RBF-SVC. Grover query counts are compared separately from wall time.

$$Q_Grover(N)=O(\sqrt{N}),\ \text{versus}\ Q_classical(N)=O(N);\ \text{evaluate runtime separately} \tag{9}$$

4.I.2 Algorithms and experimental method

DJ and BV distinguish oracle structure; Grover applies oracle and diffuser; QFT is composed of Hadamard, controlled phase rotations and swaps; QPE estimates a phase through controlled powers and inverse QFT. The tiny Shor case uses exact 16×16 controlled modular-permutation matrices. The annealing demonstration reuses Track B's Max-Cut QUBO but is classical simulated annealing, not quantum annealing. Noise, training curves, circuit evaluation count, parameter count and barren-plateau proxies are recorded.

4.I.3 Results and interpretation

The circuits returned expected results: BV recovered 101, Grover found 10, QFT round-trip returned 101, QPE returned phase 0.25, and Shor found factors 3 and 5. Grover used 1/2/3 oracle queries for N=4/8/16 versus 4/8/16 worst-case classical queries. Simulator time was roughly four orders slower than a Python linear scan at these sizes. All four QML models reached 1.00 test accuracy. VQC used 8 trainable parameters, 13,510 circuit executions and 6.81 s median; logistic regression used 3 parameters and 0.000728 s. The fixed quantum kernel used 765 circuit evaluations and 0.262 s, while RBF-SVC took 0.000227 s.

Table 12. Matched Iris quantum and classical model results

Model	Accuracy	Parameters	Circuit evaluations	Median training time
VQC / QNN	1.00	8	13,510	6.81 s
Logistic regression	1.00	3	0	0.000728 s
Quantum fidelity kernel	1.00	0 trainable	765	0.262 s
RBF SVC	1.00	6 fitted coefficients	0	0.000227 s

4.I.4 Correctness, limitations, and reproduction

The simulator confirms algorithm behavior, not practical advantage. Data encoding, circuit executions, noise and statevector cost overwhelm toy instances. Grover's theoretical oracle query complexity O(√N) versus O(N) is separate from the classical CPU simulation time. Global-cost gradient variance falls with qubit count in the tested finite range, but not enough to prove asymptotic scaling. Exact class syllabus coverage remains unresolved.

Reproduction: ./scripts/run_track.py I | .venv/bin/python -m pytest tracks/I_quantum_qml/tests

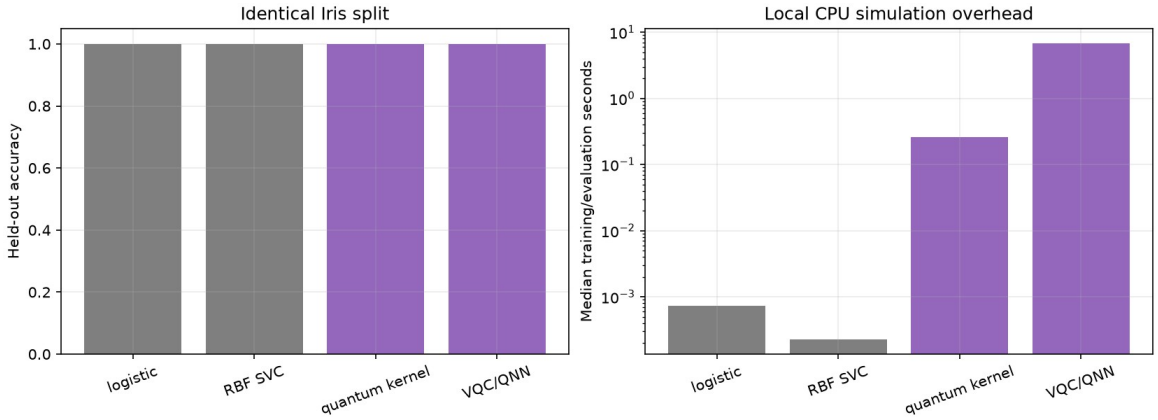


Figure 10. Track I measured comparison retained in the repository results.

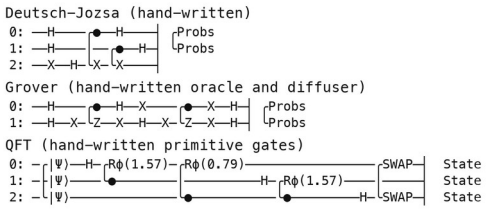


Figure 11. Hand-constructed Deutsch–Jozsa, Grover and QFT circuits from Track I.

4.J Track J Software Engineering Test Selection

4.J.1 Problem, objective, and formulation

J formulates test-suite selection on a real public Python project: MIT-licensed more-itertools at pinned commit f42daddf205f335fea6e200dad46049f33e71221. The checkout has 7,282 Python source lines and a 771-test baseline suite.

For candidate test class i, binary x_i indicates selection, t_i is measured setup/call/teardown duration, and C_i is the set of executable source lines attributed to it. Maximize $|\bigcup_{(i:x_i=1)} C_i|$ subject to $\Sigma t_i x_i \leq B$, where B is 10% of profiled candidate duration. Greedy chooses marginal new lines per second; original class order is the naive comparator.

$$\text{maximize } |\bigcup_{(i:x_i=1)} C_i|, \text{ subject to } \Sigma_i t_i x_i \leq B,\ x_i \in \{0,1\} \qquad (10)$$

4.J.2 Algorithms and experimental method

The full suite establishes correctness and the baseline; coverage contexts plus per-class durations establish the profile before selection. A 24-class deterministic sample from 167 classes forms a bounded candidate pool. An exhaustive search over 14 candidates checks the greedy method on a smaller exact instance. Each selected schedule is rerun three times outside coverage instrumentation.

4.J.3 Results and interpretation

The full 771-test suite passed three times with median wall time 7.105 s. Under 0.044501 s profiled budget, original order selected 50 tests and covered 126/271 candidate lines (46.5%); greedy selected 71 and covered 232/271 (85.6%). Both ran in about 0.133 s end-to-end, because pytest startup dominates. On a smaller exact subset, greedy matched the optimum at 86/164 lines while original order covered 61. Every selected suite passed its three verification runs.

Table 13. Test selection under a 0.044501 s profiled budget

Schedule	Selected tests	Candidate lines	Coverage	Actual wall time
Original order	50	126 / 271	46.5%	0.1330 s
Greedy coverage/s	71	232 / 271	85.6%	0.1333 s
Exact 14-class subset	27	86 / 164	52.4%	0.1122 s

4.J.4 Correctness, limitations, and reproduction

The practical result is substantially more candidate coverage for similar short-suite wall time. Context-attributed candidate coverage is not whole-project coverage; profiled component duration differs from process wall time. Selected tests omit assertions, edge cases and failure modes exercised elsewhere. Therefore the optimized suite is a change-feedback subset, not a replacement for full regression testing or release gates.

Reproduction: ./scripts/run_track.py J | .venv/bin/python -m pytest tracks/J_software_engineering/tests

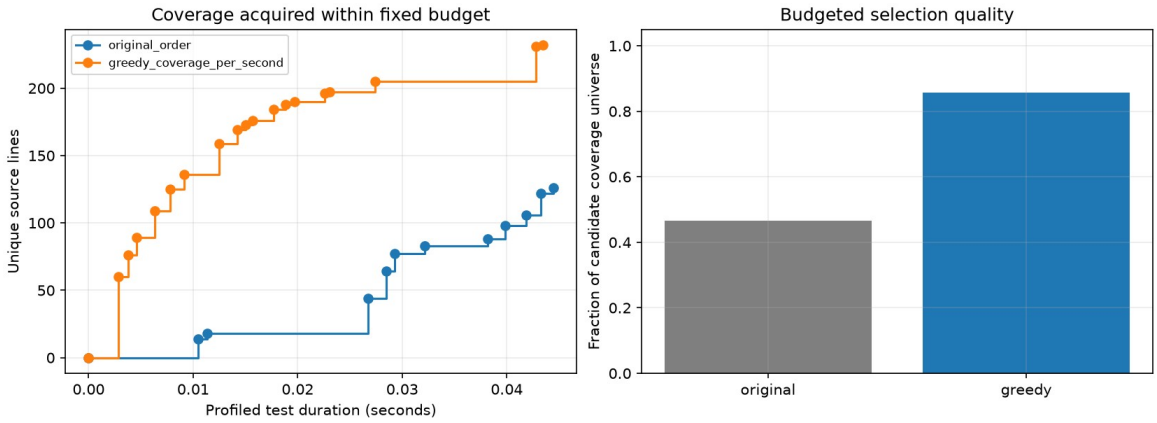


Figure 12. Track J measured comparison retained in the repository results.

5 Cross Track Analysis

Several themes recur across the results. First, exact methods are most useful as references at small scale: enumeration validates the knapsack solvers, Max-Cut Hamiltonian, submodular coverage, compiler pass order, test-selection subset and tiny batch plan. At larger scales, those methods become expensive, so the experiment reports a heuristic's verified gap or uses a bounded local surrogate.

Second, optimization can shift cost instead of removing it. Track F reduced IR size but compilation took longer. Track D's process overhead erased useful work on tiny kernels. Track E's waiting window worsened sparse-load latency. Track J gained candidate coverage at nearly identical end-to-end short-suite runtime because process startup dominated. Track G traded response equality for faster high-priority CPU completion. These are practical costs of each decision rule, not defects to conceal.

Third, resource metrics need context. High utilization in Track E accompanied a high miss rate under overload. Track H's latency reduction depended on a read-heavy query mix and 180-page storage allowance. Track A's objective values are comparable only where the

objective definition is the same; regularized and unregularized loss values cannot be compared as if identical. Quantum oracle counts express theoretical query complexity, while simulator seconds measure classical statevector work.

Fourth, the correctness boundary is track-specific: gradient finite differences; exhaustive objective checks; request conservation; semantic equivalence; xv6 usertests and checksum comparison; SQL result hashes; circuit bitstrings and matched data splits; test reruns. A method can improve its chosen objective without proving broader quality. For example, selected-test line coverage cannot guarantee detection of every fault.

Table 14. Cross-track findings and limits

Area	Measured or verified result	Interpretation boundary
Continuous optimization	Newton/BFGS reach the same regularized loss as stochastic methods with fewer steps	Tiny convex logistic model
Discrete optimization	Exact solvers beat greedy on all 3 knapsack instances	Instances are small; solver startup dominates
Parallel computing	Shared queue reaches 1.065× at 4 workers; several configurations slow down	12 fixed CPU tasks on one laptop
Batch scheduling	Greedy helps under loaded arrivals; hurts sparse latency	Simulation-only service model
Compiler	IR instruction count falls 45.2%; compile time rises 53.7%	Toy IR interpreter, not native machine code
xv6 scheduler	High-priority CPU turnaround halves; fairness index falls	One CPU, coarse QEMU ticks
SQLite	MILP/DP index sets lower weighted latency by about 88%	Synthetic read workload; cache and version dependent
Quantum	Named textbook circuits pass; classical models tie accuracy faster	No physical device; toy data and circuits
Test selection	Candidate coverage rises 46.5%→85.6% at similar wall time	Subset does not replace regression

5.1 Adaptation to larger problems

At larger scales, the next steps would preserve the same validation framework while changing only the candidate model: tune learning rates on held-out objectives; use branch-and-cut or relaxations for larger integer models; estimate batch behavior using measured service distributions; extend compiler IR with control-flow-aware dataflow; test scheduler fairness under adversarial arrivals; include index maintenance and write transactions; use device-calibrated quantum noise or hardware only where accessible; and refresh test costs/coverage on every relevant source revision. These are proposals, not results from this project.

6 AI Assisted Development and Verification

The repository register identifies OpenAI Codex as the only AI tool used from 29 October through 6 November 2026. Earlier track records expose a GPT-5 family label without an exact snapshot; the report-generation session identifies GPT-6, also without an exact deployed snapshot. Prompts, relevant responses, verification notes and record metadata are retained for cross-track planning, Tracks A–J and this report. Git history labels AI-assisted and manual commits where recorded. The report-generation prompt is recorded as this documentation task, not as earlier implementation history.

Verification was independent of generated code where practical. The workflow used finite differences for derivatives, enumeration for discrete objectives, checksum and output comparisons, process identifier checks, query-plan inspection and result hashes, regression tests, and full xv6 usertests. Tracked failures include optimizer error handling, mismatched workload generation across capacity sweeps, incorrect coverage denominators, incomplete patch generation from the wrong Git root, and plotting-axis mistakes. Each retained failure entry records its symptom, correction and regression check.

The assignment requires comparison of at least two AI tools on the same task. `logs/tool_comparison.md` explicitly records that no second tool has been used, and no second-tool responses or timing data exist. Accordingly, tool correctness, hallucination rate, speed and iteration counts cannot be compared honestly. This requirement remains open and should be completed by giving a second available assistant the same bounded task, archiving both responses, and running the same tests against each result.

Table 15. AI tool disclosure and provenance status

Requirement	Repository evidence	Status
Tool and model register	OpenAI Codex; GPT-5 family; snapshot not exposed	Recorded, exact version unknown
Track A–J prompts and verification	Per-track prompt/response/verification records	Recorded in repository
Genuine implementation failures	logs/failures.md, entries F-001 through F-022	Recorded; real fixes and regression checks
Two-tool comparison	logs/tool_comparison.md	Not performed
AI commit attribution	Git history includes [AI:Codex] and [MANUAL] labels	Partly recorded; see history
Viva readiness	report/VIVA_NOTES.md	Student explanation and live modification required

7 Conclusion

This project implemented and measured ten separate optimization studies with a shared, reproducible Python repository. The strongest gains were domain-specific: exact combinatorial solvers confirmed optima on small instances; load-aware inference batching improved the simulated overloaded workload; priority scheduling reduced high-priority CPU turnaround in xv6; SQLite's selected indexes materially reduced the measured read workload; and greedy test selection captured more candidate line coverage within the same profiled budget. Other experiments produced negative or mixed outcomes, including process overhead, sparse batching delay, longer compiler build time and fairness loss under priority scheduling.

The evidence supports only the stated scale and environment. It does not establish production performance, native compiler speedups, real LLM inference improvements or quantum advantage. The report also remains conditional on the exact Chapter 6 syllabus and a real two-tool comparison. Completing those two academic checks is necessary before describing assignment coverage as fully closed.

The central conclusion is methodological: a credible optimization result requires an explicit objective, fair baseline, constraint checks, independent correctness evidence and an honest account of overhead. Several methods increased quality only under particular load or cost assumptions; their performance reversed when those assumptions changed.

References

[1] VJTI Mumbai, Optimization Techniques, Lab Assignment 8, “Optimize Anything: The 14-Day Vibe-Coding Challenge,” supplied assignment handout, 7 pages, 2026.

[2] Bianca Sawant, bm.md, proposed minimum-implementation blueprint for Assignment 8, repository planning document, 2026.

[3] Project repository, OTLab_8: REQUIREMENTS.md, PROGRESS.md, SETUP.md, per-track source, tests, raw results, manifests, figures, prompt logs and report drafts. Accessed 6 November 2026.

[4] MIT xv6-riscv, pinned source commit fa2ddcb7ca87a77af579380a6eafd5aff163b44f. MIT License. Repository copy and patch evidence under external/xv6-riscv and tracks/G_xv6/.

[5] more-itertools, pinned source commit f42daddf205f335fea6e200dad46049f33e71221. MIT License. Provenance and license digest in tracks/J_software_engineering/SOURCE.md.

[6] SciPy, PyTorch, PennyLane, scikit-learn, SQLite and Python project versions as recorded in SETUP.md and each result manifest. These tools are used as implementations; the results remain specific to the pinned local environment.

Appendix A Chapter 6 Coverage Checklist

The assignment PDF supplies the mandatory named topics, but it also conditions several requirements on the actual methods taught in class. The supplied repository contains no matching Chapter 6 lecture notes or instructor topic list. The checklist below therefore maps all methods expressly named in the PDF while preserving unresolved syllabus-dependent rows.

Table 16. Coverage of methods expressly named by the assignment PDF

Topic	Demonstration / evidence	Status
ML optimization foundations; regularization; convex/non-convex; extrema and saddles	Track A synthetic loss, quadratic, tilted double well and regularization results	Demonstrated
GD, SGD, mini-batch, Momentum, NAG, AdaGrad, RMSProp, Adam	A method_runs.csv; shared objective and three seeds	Demonstrated
Newton, Gauss–Newton, backpropagation	A second_order_checks.json and backprop.json	Demonstrated
HVP and curvature approximations	A curvature.json; exact HVP plus diagonal/Fisher/rank-1	Named coverage present; exact syllabus unknown
Greedy, ILP, branch-and-bound, DP; QUBO/Ising and penalties	B knapsack, exhaustive Max-Cut and exactly-one penalty	Demonstrated
QAOA, VQE, PQC/QNN, parameter shift, SPSA, QNG	C experiments, tests and local simulator results	Demonstrated in simulation
Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE, Shor	I algorithm_results.csv; four primitive-gate circuits	Demonstrated at toy scale
Quantum annealing and embedding constraints	I classical annealing proxy; C chain-strength experiment	Simulation only
Variational classifier and quantum kernels	I matched Iris models, split digest and circuit counts	Demonstrated; classical accuracy ties
Finite shots, hardware noise, barren plateaus	C/I repeated shot, noise and finite-size gradient studies	Simulator/noise model only

Topic	Demonstration / evidence	Status
Submodularity and greedy approximation assumptions	D exhaustive diminishing-returns checks and 7/8 coverage	Demonstrated
Load balancing, work stealing, locality, synchronization, communication	D real processes, work-stealing simulation, overhead controls	Demonstrated; overhead included
Data, model, pipeline parallelism; Amdahl/Gustafson	D spawned variants, makespan, speedup, efficiency and law analysis	Demonstrated
FedAvg; MGDA and GradNorm where covered	D identical-start comparisons and training curves	Named methods demonstrated
Strategy selection	Track E table and repository-wide method matrix	Demonstrated
Additional methods explicitly taught in Chapter 6	Requires the actual classroom material	UNRESOLVED: audit syllabus line by line

Open syllabus checks are listed in coverage_matrix.csv and REQUIREMENTS.md as SYL-01 through SYL-08. Possible items to verify include additional quasi-Newton or curvature methods, combinatorial applications, hybrid quantum-classical algorithms, quantum algorithms and QML approaches, and distributed or multi-task methods. Their inclusion here is a question for syllabus review, not a claim that they were taught.

Appendix B Reproduction Commands and Evidence Map

Start from the repository root on the recorded macOS environment. The lockfile is authoritative for Python packages. Re-run the preflight before experiments if the machine or environment changes.

```
UV_PYTHON_INSTALL_DIR="$PWD/.python" uv sync --locked --python 3.12
./scripts/preflight.sh
./scripts/test_all.sh
./scripts/run_track.py --list
./scripts/run_track.py A   # replace A with B through J
./scripts/run_all.py
```

Primary evidence locations: tracks/<track>/implementation contains implementation; tracks/<track>/benchmarks contains runnable experiments; tracks/<track>/tests contains correctness checks; tracks/<track>/results contains raw exports, summaries, plots, environment records and manifests; report/track_<letter>.md contains detailed draft narratives; prompts/<letter>/ contains actual AI history. Track G's kernel patch and usertests logs and Track H's SQL plans are linked from their respective result folders.